

UNIT-1

Introduction to Java

Java Evolution: A Historical Perspective

- **Java History**
- java's journey began in 1991 at Sun Microsystems (later acquired by Oracle) with a project called "Green." James Gosling, Mike Sheridan, and Patrick Naughton initiated this project, aiming to develop a language for consumer electronics. The initial language, named "Oak" after an oak tree outside Gosling's window, was later renamed "Java" in 1995.
- The primary goal was to create a platform-independent language that could run on various devices. This was a significant departure from traditional languages like C and C++, which were highly dependent on the underlying operating system and hardware.

- since then, Java has undergone numerous revisions and improvements, each adding new features and capabilities. Key milestones include:
- **Java 2 (1998):** Introduced different editions like J2SE (Standard Edition), J2EE (Enterprise Edition), and J2ME (Micro Edition) to cater to different development needs.
- **Java 5 (2004):** Added significant features like generics, annotations, and auto boxing/unboxing.
- **Java 8 (2014):** Introduced lambda expressions and the Stream API, revolutionizing functional programming in Java.
- **Java 11 (2018):** Introduced modularity and streamlined the development process.

Key Features of Java

- **Platform Independence:** Java's "write once, run anywhere" (WORA) principle is a cornerstone of its design. Java code is compiled into bytecode, which is then executed by the Java Virtual Machine (JVM). The JVM acts as an intermediary between the bytecode and the underlying operating system, ensuring that the same Java code can run on any platform with a JVM.
- **Object-Oriented:** Java is a fully object-oriented programming (OOP) language. It supports key OOP concepts like encapsulation, inheritance, and polymorphism, enabling developers to create modular, reusable, and maintainable code.

- **Simple and Familiar:** Java's syntax is relatively simple and similar to C and C++, making it easier for programmers familiar with these languages to learn Java.
- **Robust:** Java incorporates features like automatic garbage collection and exception handling to prevent memory leaks and handle runtime errors gracefully, making it a robust language.
- **Secure:** Java provides a secure environment for executing code. The JVM includes security features like byte code verification and security managers to prevent malicious code from harming the system.

- **Multithreaded:** Java supports multithreading, allowing developers to create applications that can perform multiple tasks concurrently, improving performance and responsiveness.
- **Distributed:** Java supports distributed computing, enabling developers to create applications that can run on multiple computers and communicate with each other over a network.

- **Dynamic:** Java is a dynamic language, allowing code to be loaded and linked at runtime. This flexibility makes it suitable for developing complex and evolving applications.
- **High Performance:** While initially slower than compiled languages like C++, Java's performance has improved significantly over the years due to advancements in JVM technology, such as Just-In-Time (JIT) compilation.

Java vs. C and C++: Key Differences

Java vs. C/C++ Features

Feature	Java	C/C++
Platform	Platform Independent (WORA)	Platform Dependent
Memory Management	Automatic Garbage Collection	Manual Memory Management
Pointers	No explicit pointers	Explicit pointers are used
Inheritance	Single/Interface	Multiple
Operator Overloading	Not supported	Supported
Data Types	Primitive/Reference	Primitive/Derived/User-defined
Security	Built-in	Programmer's responsibility

Java and the Internet

- Java's emergence coincided with the rise of the internet, and its features were well-suited for web development. Java applets, small programs embedded in web pages, allowed for interactive and dynamic content. While applets are less common today, Java continues to play a crucial role in web development through technologies like Java servlets, JavaServer Pages (JSP), and frameworks like Spring and Jakarta EE

Java and the World Wide Web

- Java's platform independence and security features made it an ideal language for developing web applications. Java's ability to run on different web servers and interact with databases made it a popular choice for building enterprise-level web applications.

Web Browsers and Java

- Early web browsers like Netscape Navigator and Internet Explorer included Java Virtual Machines (JVMs) to run Java applets. However, due to security concerns and the rise of other web technologies like JavaScript, support for Java applets has been phased out in modern browsers.

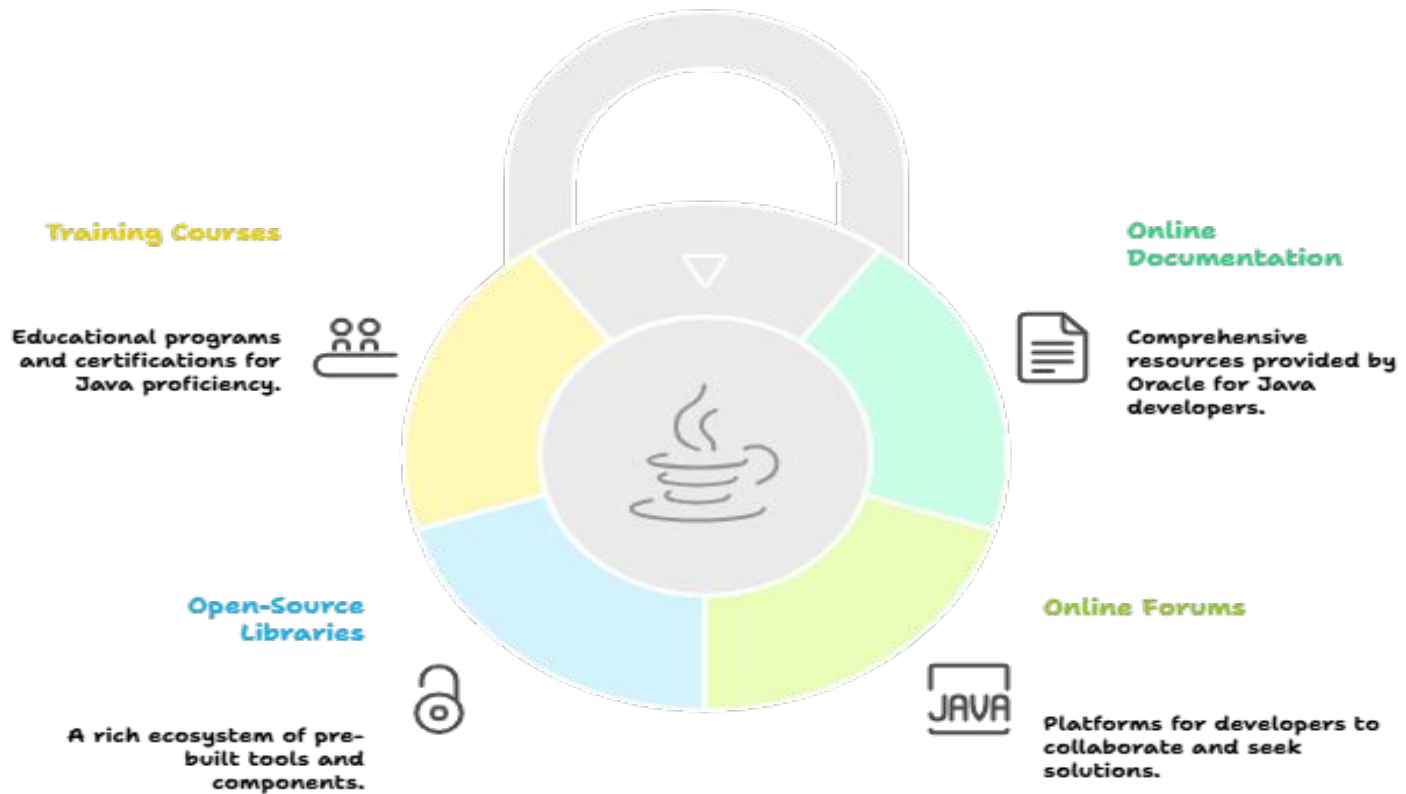
Hardware and Software Requirements

- **Hardware:** A computer with a processor, memory (RAM), and storage (hard drive or SSD). The specific requirements depend on the complexity of the applications you intend to develop.
- **Software:**
 - **Java Development Kit (JDK):** Contains the tools necessary to compile, debug, and run Java applications.
 - **Integrated Development Environment (IDE):** Provides a user-friendly interface for writing, compiling, and debugging Java code. Popular IDEs include Eclipse, IntelliJ IDEA, and NetBeans.
 - **Java Runtime Environment (JRE):** Provides the necessary libraries and the JVM to run Java applications. The JRE is included in the JDK.

Java Support Systems

- Java has a vibrant and active community that provides extensive support for developers:
- **Online Documentation:** Oracle provides comprehensive documentation for the Java platform, including API documentation, tutorials, and articles.
- **Online Forums and Communities:** Websites like Stack Overflow, Reddit (r/java), and various Java user groups provide platforms for developers to ask questions, share knowledge, and collaborate.
- **Open-Source Libraries and Frameworks:** A vast ecosystem of open-source libraries and frameworks is available for Java, providing pre-built components and tools for various development tasks.
- **Training Courses and Certifications:** Numerous training courses and certifications are available for Java, ranging from beginner to advanced levels.

Java Developer Support System



The Java Environment

- **java Development Kit (JDK):** A software development kit that includes the tools needed to develop, compile, and debug Java applications.
- **Java Runtime Environment (JRE):** Provides the runtime environment for executing Java applications.
- **Java Virtual Machine (JVM):** An abstract computing machine that executes Java bytecode.
- **Java API (Application Programming Interface):** A collection of pre-built classes and interfaces that provide functionality for various tasks, such as input/output, networking, and GUI development.

Introduction to Java language

- Java is a high-level, class-based, object-oriented programming language that is designed to have as few implementation dependencies as possible. It is a general-purpose programming language intended to let application developers write once, run anywhere (WORA), meaning that compiled Java code can run on all platforms that support Java without the need for recompilation. Java applications are typically compiled to bytecode that can run on any Java virtual machine (JVM) regardless of the underlying computer architecture.

- Java is widely used for developing enterprise applications, mobile applications (particularly Android), web applications, and desktop applications. Its robustness, security features, and platform independence make it a popular choice for a wide range of projects.

Simple Java Program

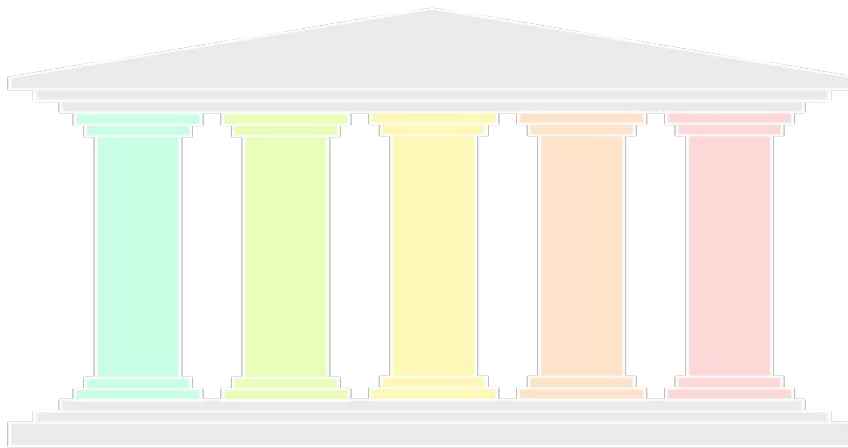
```
public class HelloWorld {  
    public static void main(String[] args) {  
        System.out.println("Hello, World!");  
    }  
}
```

This program defines a class named HelloWorld. The main method is the entry point of the program. The `System.out.println()` statement prints the text "Hello, World!" to the console.

More of Java

- Java is an object-oriented language, which means it revolves around the concept of objects and classes. Classes are blueprints for creating objects. Objects are instances of classes.

Core Concepts of Java



Classes and Objects

Templates for creating objects with state and behavior.



Inheritance

Allows classes to inherit properties and methods.



Polymorphism

Treating objects of different classes as a common type.



Encapsulation

Bundling data and methods within a class.



Abstraction

Representing essential features without details.

Application with Two Classes

```
class Dog {  
    String name;  
    public Dog(String name) {  
        this.name = name;  
    }  
    public void bark() {  
        System.out.println(name + " says Woof!");  
    }  
}  
  
public class Main {  
    public static void main(String[] args) {  
        Dog myDog = new Dog("Buddy");  
        myDog.bark();  
    }  
}
```

Java Program Structure

A typical Java program consists of one or more classes. Each class contains fields (variables) and methods (functions). The basic structure is as follows:

```
// Package declaration (optional)
package com.example;

// Import statements (optional)
import java.util.ArrayList;

// Class definition
public class MyClass {
    // Fields (variables)
    int myVariable;

    // Constructor
    public MyClass(int value) {
        this.myVariable = value;
    }

    // Methods (functions)
    public void myMethod() {
        System.out.println("Value: " + myVariable);
    }

    // Main method (entry point) - optional, only in classes that start the
    program
}
```

```
// Main method (entry point) - optional, only in classes that start the
program
public static void main(String[] args) {
    MyClass myObject = new MyClass(10);
    myObject.myMethod();
}
}
```

Java Tokens

- java tokens are the smallest individual units in a Java program. They include:
- **Keywords:** Reserved words with special meanings (e.g., class, public, static, void, int, if, else, while).
- **Identifiers:** Names given to variables, methods, classes, etc. (e.g., myVariable, myMethod, MyClass).
- **Literals:** Constant values (e.g., 10, 3.14, "Hello").
- **Operators:** Symbols that perform operations (e.g., +, -, *, /, =, ==).
- **Separators:** Symbols that separate elements of the code (e.g., ;, {}, (), []).

Java Statements

Java statements are instructions that the Java compiler executes. They can be simple statements or compound statements.

- **Declaration Statements:** Declare variables (e.g., `int x;`).
- **Assignment Statements:** Assign values to variables (e.g., `x = 10;`).
- **Control Flow Statements:** Control the flow of execution (e.g., `if`, `else`, `for`, `while`).
- **Method Call Statements:** Call methods (e.g., `System.out.println("Hello");`).
- **Return Statements:** Return values from methods (e.g., `return x;`).

Implementing a Java Program

- Implementing a Java program involves the following steps:
- **Writing the Code:** Write the Java code in a text editor or an Integrated Development Environment (IDE).
- **Compiling the Code:** Use the javac compiler to compile the .java files into .class files (byte code).
- **Running the Code:** Use the java command to run the compiled .class files on the JVM.

Java Virtual Machine (JVM)

The Java Virtual Machine (JVM) is a runtime environment that executes Java byte code. It provides platform independence by abstracting away the underlying hardware and operating system. The JVM performs tasks such as:

- **Loading Byte code:** Loading the .class files into memory.
- **Verifying Byte code:** Ensuring the byte code is valid and secure.
- **Executing Byte code:** Interpreting or compiling the byte code into machine code and executing it.
- **Garbage Collection:** Automatically managing memory by reclaiming unused objects.

Command Line Arguments

Java programs can accept command-line arguments, which are passed to the main method as an array of strings.

```
public class CommandLineArgs {  
    public static void main(String[] args) {  
        for (int i = 0; i < args.length; i++) {  
            System.out.println("Argument " + i + ": " + args[i]);  
        }  
    }  
}
```

To run this program with command-line arguments:

```
java CommandLineArgs arg1 arg2 arg3
```

The output will be:

```
Argument 0: arg1  
Argument 1: arg2  
Argument 2: arg3
```

Programming Style

Good programming style is essential for writing maintainable and readable code. Some common Java programming style conventions include:

- **Indentation:** Use consistent indentation to improve readability.
- **Naming Conventions:** Use meaningful names for variables, methods, and classes.
- **Comments:** Add comments to explain the purpose of the code.
- **Code Organization:** Organize code into logical blocks and use blank lines to separate them.
- **Avoid Magic Numbers:** Use named constants instead of hardcoded values.

Constants

Constants are values that cannot be changed during the execution of a program. In Java, constants are declared using the final keyword.

EXAMPLE:

```
final int MAX_VALUE = 100;  
final double PI = 3.14159;  
final String GREETING = "Hello";
```

Variables

Variables are named storage locations that can hold values. In Java, variables must be declared with a specific data type.

`int age;` // Integer variable

`double salary;` // Double-precision floating-point variable

`String name;` // String variable

`Boolean is Employed;` // Boolean variable

Data Types

Primitive data types are the basic building blocks of data representation in Java. They are predefined and directly hold values. Java has eight primitive data types:

- **byte:** 8-bit signed integer. Range: -128 to 127.
- **short:** 16-bit signed integer. Range: -32,768 to 32,767.
- **int:** 32-bit signed integer. Range: -2,147,483,648 to 2,147,483,647.
- **long:** 64-bit signed integer. Range: -9,223,372,036,854,775,808 to 9,223,372,036,854,775,807.
- **float:** 32-bit single-precision floating-point number.
- **double:** 64-bit double-precision floating-point number.
- **char:** 16-bit Unicode character. Represents a single character.
- **Boolean:** Represents a truth value, either true or false.

Reference Data Types

- Reference data types, also known as non-primitive data types, do not directly store values. Instead, they hold references (memory addresses) to objects. Examples of reference data types include:
 - **Classes:** User-defined blueprints for creating objects.
 - **Interfaces:** Contracts that define a set of methods that a class must implement.
 - **Arrays:** Collections of elements of the same data type.
 - **Strings:** Sequences of characters (technically an object of the String class).

Variable Declaration Rules in Java

- **Every variable must be declared with a data type** before it can be used. For example, `int age;` declares a variable named `age` that stores integers.
- **Variable names must start with a letter (A–Z or a–z), an underscore (_), or a dollar sign (\$).** For example, `_total` and `$value` are valid variable names.
- **Variable names cannot start with a digit.** For example, `1num` is invalid.
- **After the first character, variable names can contain letters, digits, underscores, or dollar signs.** For example, `value1` is valid.
- **You cannot use Java reserved keywords as variable names.** For example, `int class;` is invalid because `class` is a keyword.
- **Java is case-sensitive, meaning variable names like `num`, `Num`, and `NUM` are all different variables.**
- **Variable names should be meaningful and descriptive.** Instead of using vague names like `a` or `sm`, use names like `age` or `studentMarks` to make your code more readable.

Declaration of Variables

- Before using a variable, it must be declared. The declaration specifies the variable's data type and name.

The syntax for declaring a variable is:

data_Type variable_Name;

For example:

- `int age;`
- `double salary;`
- `String name;`

Giving Values to Variables

- After declaring a variable, you can assign a value to it using the assignment operator (=).

Variable Name = value;

For example:

- `age = 25;`
- `salary = 50000.0;`
- `name = "John Doe";`

Scope of Variables

The scope of a variable determines where in the program the variable can be accessed. Java has different types of variable scopes:

- **Local Variables:** Declared inside a method or block of code. They are only accessible within that method or block.
- **Instance Variables:** Declared inside a class but outside any method. They belong to an instance of the class (an object) and are accessible from any method within the class.
- **Static Variables:** Declared inside a class with the static keyword. They belong to the class itself and are shared by all instances of the class.

Symbolic Constants

Symbolic constants are named constants that are assigned a value only once. They are declared using the final keyword. Once a value is assigned to a symbolic constant, it cannot be changed.

EXAMPLE :

```
final double PI = 3.14159;
```

Type Casting

Type casting is the process of converting a value from one data type to another. Java supports two types of type casting:

1.Implicit Casting (Widening): Automatically converts a value from a smaller data type to a larger data type without loss of information.
For example, converting an int to a double.

```
`int x = 10;
```

```
double y = x; // Implicit casting from int to  
double``
```

Explicit Casting (Narrowing):

Converts a value from a larger data type to a smaller data type. This requires using a cast operator and may result in loss of information.

For example,

converting a double to an int.

```
double x = 10.5; int y = (int) x;
```

```
// Explicit casting from double to int (y will be  
10)
```

Getting Values of Variables

The values of variables can be accessed directly by using their names. You can also use methods like Scanner to get input from the user and assign it to variables.

```
import java.util.Scanner;

public class InputExample {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        System.out.print("Enter your name: ");
        String name = scanner.nextLine();

        System.out.print("Enter your age: ");
        int age = scanner.nextInt();

        System.out.println("Hello, " + name + "! You are " + age + " years
old.");

        scanner.close();
    }
}
```

Standard Default Values

- When a variable is declared but not explicitly initialized, Java assigns a default value to it based on its data type:
- **byte, short, int, long:** 0
- **float:** 0.0f
- **double:** 0.0d
- **char:** '\u0000' (null character)
- **boolean:** false
- **Reference Types (e.g., String, Objects):** null
- Understanding these default values is important for avoiding unexpected behavior in your programs.

Operators and Expressions in Java:

This document provides a detailed overview of operators and expressions in Java, covering various types of operators including arithmetic, relational, logical, assignment, increment/decrement, conditional, bitwise, and special operators. It also delves into arithmetic expressions, their evaluation, operator precedence, type conversion, associativity, and commonly used mathematical functions in Java.

Arithmetic Operators

Arithmetic operators are used to perform mathematical calculations. Java supports the following arithmetic operators:

Operator	Description	Example
----------	-------------	---------

+	Addition	$a + b$
-	Subtraction	$a - b$
*	Multiplication	$a * b$
/	Division	a / b
%	Modulus (Remainder)	$a \% b$

Example:

```
int a = 10;  
int b = 5;  
  
System.out.println("a + b = " + (a + b)); // Output: a + b = 15  
System.out.println("a - b = " + (a - b)); // Output: a - b = 5  
System.out.println("a * b = " + (a * b)); // Output: a * b = 50  
System.out.println("a / b = " + (a / b)); // Output: a / b = 2  
System.out.println("a % b = " + (a % b)); // Output: a % b = 0
```

Relational Operators

Relational operators are used to compare two operands and determine the relationship between them. They return a boolean value (true or false).

Operator	Description	Example
1. ==	Equal to	a == b
2. !=	Not equal to	a != b
3. >	Greater than	a > b
4. <	less than	a < b
5. >=	Greater than or equal to	a >= b
6. <=	Less than or equal to	a <= b

Example:

```
int a = 10;
int b = 5;
```

```
System.out.println("a == b: " + (a == b)); // Output: a == b: false
System.out.println("a != b: " + (a != b)); // Output: a != b: true
System.out.println("a > b: " + (a > b));    // Output: a > b: true
System.out.println("a < b: " + (a < b));    // Output: a < b: false
System.out.println("a >= b: " + (a >= b)); // Output: a >= b: true
System.out.println("a <= b: " + (a <= b)); // Output: a <= b: false
```

Logical Operators

- Logical operators are used to combine two or more conditions and return a Boolean value.

Operator	Description	Example
----------	-------------	---------

1. &&	Logical AND (returns true if both are true)	a && b
----------	---	--------

2.	Logical OR (returns true if either is true)	a b
----	---	--------

3. !	Logical NOT (reverses the Boolean value)	!a
------	--	----

Example:

```
boolean a = true;  
boolean b = false;  
  
System.out.println("a && b: " + (a && b)); // Output: a && b: false  
System.out.println("a || b: " + (a || b)); // Output: a || b: true  
System.out.println("!a: " + (!a));        // Output: !a: false
```

Assignment Operators

Assignment operators are used to assign a value to a variable. The most common assignment operator is =. Java also provides shorthand assignment operators.

Operator | Description | Example | Equivalent |

1.	=	Assignment	a = b	a = b
2.	+=	Add and assign	a += b	a = a + b
3.	-=	Subtract and assign	a -= b	a = a - b
4.	*=	Multiply and assign	a *= b	a = a * b
5.	/=	Divide and assign	a /= b	a = a / b
6.	%=	Modulus and assign	a %= b	a = a % b

Example:

```
int a = 10;  
int b = 5;  
  
a += b; // a = a + b = 15  
System.out.println("a: " + a); // Output: a: 15  
  
a -= b; // a = a - b = 10  
System.out.println("a: " + a); // Output: a: 10
```

Increment and Decrement Operators

Increment and decrement operators are used to increase or decrease the value of a variable by 1.

- **Operator | Description | Example |**

++	Increment	a++ or ++a
--	Decrement	a-- or --a

prefix vs. Postfix:

- **Prefix:** The operator is placed before the variable (++a or --a). The value of the variable is incremented/decremented first, and then the new value is used in the expression.
- **Postfix:** The operator is placed after the variable (a++ or a--). The current value of the variable is used in the expression, and then the variable is incremented/decremented.

Example:

```
int a = 5;
```

```
System.out.println("a++: " + (a++)); // Output: a++: 5 (a becomes 6 after this  
line)
```

```
System.out.println("a: " + a);      // Output: a: 6
```

```
int b = 5;
```

```
System.out.println("++b: " + (++b)); // Output: ++b: 6 (b becomes 6 before this  
line)
```

```
System.out.println("b: " + b);      // Output: b: 6
```

Conditional Operator (Ternary Operator)

The conditional operator is a shorthand way of writing an if-else statement. It has the following syntax:

condition ? expression1 : expression2

- If the condition is true, expression1 is evaluated; otherwise, expression2 is evaluated.

- **example:**
- `int a = 10;`
- `int b = 5;`
- `int max = (a > b) ? a : b;`
`System.out.println("Max: " + max);`
- `// Output: Max: 10`

Bitwise Operators

Bitwise operators perform operations on individual bits of integers.

Operator | Description | Example

- `&` Bitwise AND `a & b`
- `|` Bitwise OR `a | b`
- `^` Bitwise XOR `a ^ b`
- `~` Bitwise NOT (One's complement) `~a`
- `<<` Left shift `a << b`
- `>>` Right shift (signed) `a >> b`
- `>>>` Right shift (unsigned or zero-fill right shift)
`a >>> b`

Example:

```
int a = 5; // 00000101
```

```
int b = 3; // 00000011
```

```
System.out.println("a & b: " + (a & b)); // Output: a & b: 1 (00000001)
```

```
System.out.println("a | b: " + (a | b)); // Output: a | b:
```

Evaluation of Expressions

- Java evaluates arithmetic expressions according to a specific order of operations, often remembered by the acronym PEMDAS or BODMAS:
- **Parentheses/Brackets:** Expressions within parentheses or brackets are evaluated first.
- **Exponents/Orders:** Exponentiation (not directly supported by an operator in Java, but achieved through `Math.pow()`) is evaluated next.
- **Multiplication and Division:** Multiplication and division are performed from left to right.
- **Addition and Subtraction:** Addition and subtraction are performed from left to right.

Example:

```
int result = 10 + 5 * 2; // Multiplication is performed before addition  
System.out.println(result); // Output: 20
```

```
int result2 = (10 + 5) * 2; // Parentheses change the order of operations  
System.out.println(result2); // Output: 30
```

Associativity

- When operators have the same precedence, associativity determines the order in which they are evaluated. Most arithmetic operators in Java are left-associative, meaning they are evaluated from left to right.

EXAMPLE

- `int result = 10 - 5 - 2; // Subtraction is left-associative`
`System.out.println(result); //`
Output: 3 ((10 - 5) - 2)

Mathematical Functions

- Java provides a rich set of mathematical functions through the `Math` class, which is part of the `java.lang` package (and therefore automatically imported). These functions allow you to perform more complex mathematical operations.

Commonly Used Mathematical Functions:

- `Math.abs(x)` : Returns the absolute value of `x` .
- `Math.sqrt(x)` : Returns the square root of `x` .
- `Math.pow(x, y)` : Returns `x` raised to the power of `y` .
- `Math.round(x)` : Returns the closest `long` to the argument, with ties rounding up.
- `Math.ceil(x)` : Returns the smallest [closest to negative infinity] `double` value that is greater than or equal to the argument and is equal to a mathematical integer.
- `Math.floor(x)` : Returns the largest [closest to positive infinity] `double` value that is less than or equal to the argument and is equal to a mathematical integer.
- `Math.max(x, y)` : Returns the greater of `x` and `y` .
- `Math.min(x, y)` : Returns the smaller of `x` and `y` .
- `Math.random()` : Returns a pseudorandom `double` greater than or equal to `0.0` and less than `1.0` .
- `Math.sin(x)` : Returns the trigonometric sine of an angle.
- `Math.cos(x)` : Returns the trigonometric cosine of an angle.
- `Math.tan(x)` : Returns the trigonometric tangent of an angle.
- `Math.log(x)` : Returns the natural logarithm [base e] of `x` .
- `Math.log10(x)` : Returns the base 10 logarithm of `x` .

Example:

```
double number = 16.0;
double squareRoot = Math.sqrt(number);
System.out.println("Square root of " + number + " is: " + squareRoot); //
Output: Square root of 16.0 is: 4.0
```

```
double power = Math.pow(2, 3);
System.out.println("2 raised to the power of 3 is: " + power); // Output: 2
raised to the power of 3 is: 8.0
```

```
double randomNumber = Math.random();
System.out.println("Random number: " + randomNumber); // Output: Random number:
(a number between 0.0 and 1.0)
```

Decision Making and Branching

- Decision-making in programming involves evaluating conditions and executing different code blocks based on whether those conditions are true or false. Branching refers to the process of selecting a specific path of execution based on the outcome of a decision.

1. `if` Statement

The if statement is the most basic decision-making construct in Java. It allows you to execute a block of code only if a specified condition is true.

The syntax of a simple if statement is as follows:

```
if (condition) {  
    // Code to be executed if the condition is true  
}
```

EXAMPLE

```
int age = 20;  
if (age >= 18) {  
    System.out.println("You are eligible to vote.");  
}
```

2. if...else Statement

- The if...else statement allows you to execute one block of code if a condition is true and another block of code if the condition is false.
- **The syntax of an if...else statement is as follows:**

```
if (condition) {  
    // Code to be executed if the condition is true  
} else {  
    // Code to be executed if the condition is false  
}
```

Example:

```
int number = 10;  
if (number % 2 == 0) {  
    System.out.println("The number is even.");  
} else {  
    System.out.println("The number is odd.");  
}
```

3. Nesting of if...else Statements

- if...else statements can be nested within each other to create more complex decision-making logic.

Example:

```
int age = 20;
boolean hasLicense = true;

if (age >= 18) {
    if (hasLicense) {
        System.out.println("You are eligible to drive.");
    } else {
        System.out.println("You are eligible to drive, but you need a
license.");
    }
} else {
    System.out.println("You are not eligible to drive.");
}
```

4. The else if Ladder

- The else if ladder allows you to check multiple conditions in a sequence.
- **The syntax of an else if ladder is as follows:**

```
if (condition1) {  
    // Code to be executed if condition1 is true  
} else if (condition2) {  
    // Code to be executed if condition2 is true  
} else if (condition3) {  
    // Code to be executed if condition3 is true  
} else {  
    // Code to be executed if none of the conditions are true  
}
```

EXAMPLE

Example:

```
int marks = 75;

if (marks >= 90) {
    System.out.println("Grade: A");
} else if (marks >= 80) {
    System.out.println("Grade: B");
} else if (marks >= 70) {
    System.out.println("Grade: C");
} else if (marks >= 60) {
    System.out.println("Grade: D");
} else {
    System.out.println("Grade: F");
}
```

The switch Statement

- The switch statement allows you to select one of several code blocks to execute based on the value of a variable.
- **The syntax of a switch statement is as follows:**

```
switch (variable) {  
    case value1:  
        // Code to be executed if variable == value1  
        break;  
    case value2:  
        // Code to be executed if variable == value2  
        break;  
    case value3:  
        // Code to be executed if variable == value3  
        break;  
    default:  
        // Code to be executed if variable does not match any of the cases  
}  
}
```

Example:

```
int day = 3;

switch (day) {
    case 1:
        System.out.println("Monday");
        break;
    case 2:
        System.out.println("Tuesday");
        break;
    case 3:
        System.out.println("Wednesday");
        break;
    case 4:
        System.out.println("Thursday");
        break;
    case 5:
        System.out.println("Friday");
        break;
    case 6:
        System.out.println("Saturday");
        break;
    case 7:
        System.out.println("Sunday");
        break;
    default:
        System.out.println("Invalid day");
}
```



Decision Making and Looping

Looping allows you to execute a block of code repeatedly until a certain condition is met. Java provides several constructs for implementing looping logic.

1.The `while` Statement

The while statement allows you to execute a block of code repeatedly as long as a specified condition is true.

The syntax of a while statement is as follows:

The syntax of a while statement is as follows:

```
while (condition) {  
    // Code to be executed repeatedly as long as  
    the condition is true  
}
```

Example:

```
int i = 0;  
while (i < 10) {  
    System.out.println(i);  
    i++;  
}
```

The `do` Statement

The do statement is similar to the while statement, but it guarantees that the code block will be executed at least once, even if the condition is initially false.

- **The syntax of a do statement is as follows:**

```
do { // Code to be executed repeatedly as long as  
    the condition is true  
} while (condition);
```

Example:

```
int i = 0;  
do {  
    System.out.println(i);  
    i++;  
} while (i < 10);
```

In this example, the code inside the `do` loop will be executed at least once, even

The `for` Statement

The for statement is a powerful and versatile looping construct in Java used to execute a block of code repeatedly for a predetermined number of times. It provides a concise way to initialize, test, and update a loop counter within a single statement.

- **Syntax**

The basic syntax of the for statement is as follows:

for

(initialization;condition;increment/decrement) {

// Code to be executed repeatedly

}

- **Initialization:** This part is executed only once at the beginning of the loop. It typically initializes a loop counter variable.
- **Condition:** This is a Boolean expression that is evaluated before each iteration of the loop. If the condition is true, the loop body is executed. If the condition is false, the loop terminates.
- **Increment/Decrement:** This part is executed after each iteration of the loop. It typically updates the loop counter variable, either incrementing or decrementing it.

- **Example**

```
for (int i = 0; i < 10; i++) {  
    System.out.println("Value of i: " + i);  
}
```

In this example:

- `int i = 0;` initializes the loop counter `i` to 0.
- `i < 10;` is the condition that checks if `i` is less than 10. The loop continues as long as this condition is true.
- `i++;` increments the value of `i` by 1 after each iteration.
- This loop will print the values of `i` from 0 to 9.

Jumps in Loops: `break` and `continue`

- Java provides two jump statements, `break` and `continue`, that can be used to alter the normal flow of execution within a loop.
- **`break` Statement**
- The `break` statement is used to terminate the loop prematurely. When a `break` statement is encountered inside a loop, the loop is immediately exited, and the program control transfers to the next statement after the loop.

```
for (int i = 0; i < 10; i++) {  
    if (i == 5) {  
        break; // Terminate the loop when i is 5  
    }  
    System.out.println("Value of i: " + i);  
}  
System.out.println("Loop terminated.");
```

In this example, the loop will print the values of `i` from 0 to 4. When `i` becomes 5, the `break` statement is executed, and the loop terminates. The output will be:

```
Value of i: 0  
Value of i: 1  
Value of i: 2  
Value of i: 3  
Value of i: 4  
Loop terminated.
```

Labelled Loops

- In nested loops, it can be useful to control which loop a break or continue statement affects. This is achieved using labelled loops. A labelled loop is a loop that has a label associated with it. The label is an identifier followed by a colon (:) placed before the loop.

- **Syntax**

```
label: for (initialization; condition;  
    increment/decrement) {  
    // Code to be executed repeatedly  
    // break label;  
    // continue label;  
}
```

Example

```
outerLoop:ss|
for (int i = 0; i < 3; i++) {
    innerLoop:
    for (int j = 0; j < 3; j++) {
        if (i == 1 && j == 1) {
            break outerLoop; // Terminate the outer loop
        }
        System.out.println("i: " + i + ", j: " + j);
    }
}
System.out.println("Loop terminated.");
```

- In this example, the outer Loop is labelled as outer Loop, and the inner Loop is labelled as inner Loop. When i is 1 and j is 1, the break outer Loop; statement is executed, which terminates the outer Loop. The output will be:
- i: 0, j: 0 i: 0, j: 1 i: 0, j: 2 i: 1, j: 0 Loop terminated.
- If we used break inner Loop; instead, only the inner loop would terminate, and the outer loop would continue.
- Similarly, continue outer Loop; would skip the rest of the current iteration of the outer loop and proceed to the next iteration of the outer loop.